Android Display
使用指南

版本号: 1.3
发布日期: 2023.03.08

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2020.11.10	Allwinner	建立该文档
1.1	2022.05.05	AWA1639	新增 displayd 等描述，适配其他平台
1.2	2022.05.23	AWA1639	适配 T507，增加多显配置方法
1.3	2023.03.08	AWA1639	适配 android13，优化部分描述

广州星翼电子科技有限公司zdyzlinux

目　录

1 概述 **1**
1.1 编写目的 1
1.2 适用范围 1
1.3 适用人员 1

2 HWC 使用说明 **2**
2.1 HWC 代码介绍 2
2.2 hwcdebug 3
2.2.1 调试打印控制 4
2.2.2 图层数据保存 4

3 其他 display HAL 代码介绍 **5**
3.1 CEC hal 5
3.2 显示输出配置 5
3.3 显示动态配置接口实现 6
3.4 PQ 守护进程 7
3.5 hdmi edid 库 7
3.6 displayd 7

4 Android 显示性能调试简介 - systrace **8**
4.1 使用方法 8
4.1.1 离线抓取 8
4.1.2 查看支持的事件 8
4.2 信息解读 9
4.2.1 CPU 运行状态 9
4.2.2 进程运行状态 10
4.2.3 SurfaceFlinger 11
4.2.3.1 VSYNC 12
4.2.3.2 fence state 12
4.2.3.3 Queued-Buffer 数量 13
4.2.4 composer 13
4.2.5 disp_trace 14

版权所有 © 珠海全志科技股份有限公司。保留一切权利

插　图

图 2-1　HWC architecture 3
图 4-1　CPU Freq Info 10
图 4-2　Process running state 11
图 4-3　SurfaceFlinger VSYNC 12
图 4-4　Fence not signal 12
图 4-5　GPU completion 13
图 4-6　systrace composer info 14
图 4-7　disp2 trace 14

广州星翼电子科技有限公司zdyzlinux

1 概述

1.1 编写目的

让显示 Framework 开发人员了解 Android display HAL 的代码结构以及调试手段，能够快速开展调试排查工作。

1.2 适用范围

本模块设计适用于 Android 13 平台。

1.3 适用人员

与显示相关的 Framework/HAL 开发人员，与显示相关的其他模块的开发人员。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 HWC 使用说明

2.1 HWC 代码介绍

HWC 代码位于：android/hardware/aw/display/hwc-hal，主要用于实现硬件合成器相关功能对接。

目录结构如下：

```
.
├── aidl
├── common
├── disp2
│   └── include
├── include
├── rotator
├── scripts
├── strategy
├── tests
│   └── layertest
│       └── cases
├── vendorservice
│   ├── default
│   └── homlet
└── writeback
```

- aidl: 顶层适配代码，对接 HWC AIDL
- common: 通用代码，适配 android.hardware.graphics.composer
- disp2: 驱动适配代码
- rotator: 硬件旋转代码实现
- strategy: 图层合成策略
- tests: 单元测试代码，主要包括图层分配策略测试已经硬件旋转功能测试
- vendorservice：AW 自定义显示接口实现

版权所有 © 珠海全志科技股份有限公司。保留一切权利

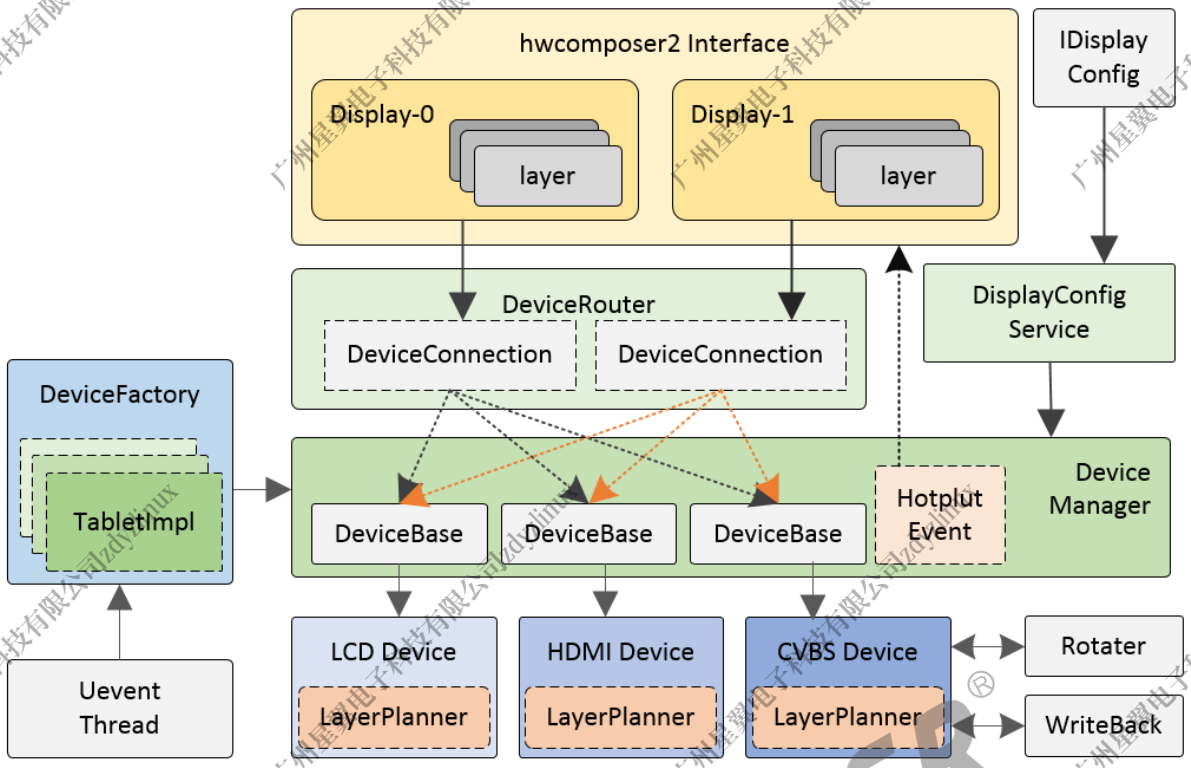

图 2-1: HWC architecture

2.2 hwcdebug

系统内集成了 HWC 调试工具 hwcdebug，通过该工具执行调试：

```
usage: hwcdebug [options] [categories...]
options include:
  -o    enable hwc log output which indicate by the
        following categories
  -x    disbale all hwc log output
  --list  list the available logging categories
  --dump <type> <count> [z]
        request dump buffer raw data into file
        type: 0 - input  buffer
              1 - output buffer
              2 - hardware rotated buffer
        count: total request count
        z : which z order of input buffer will be dump
  --freeze <enable> -s <display>
        freezes the output content of the specified logic display for debugging
  --help  show this usage
```

调试功能包括：调试信息打开/关闭，保存图层信息，保存硬件旋转输出图像信息等。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.2.1 调试打印控制

HWC 通过 tag 进行过滤调试信息输出，支持的 tag 包括：

```
 layer - Layer planner debug output
 vsync - Vsync timestamp print
 scale - Hardware scaler detect
  fps - Show primary display refresh rate
rotate - Hardware rotator relate information
 perf - Hardware rotator relate information
```

- 打开帧率信息显示

```
hwcdebug -o fps && logcat -s sunxihwc
```

- 打开图层分配信息显示

```
hwcdebug -o layer && logcat -s sunxihwc
```

2.2.2 图层数据保存

hwcdebug 目前支持截取输入图层，和硬件旋转后输出的图层，命令格式如下：

```
hwcdebug --dump <type> <count> [z]
type  - 可取值为 0/2, 0 对应输入图层，2对应硬件旋转输出图层
count - 截取图层的帧数
z    - 指定输入图层的zorder（即特定zorder的图层才会被截取），如果z=255，截取所有的输入图层
```

图层文件保存于 /data/ 目录下，以 dump_format_widthxheight_framenumber.dat 格式命名。

举例：截取 zorder 为 1 的输入图层 3 帧：

```
$ hwcdebug --dump 0 3 1
$ ls -l /data/
-rw-------  1 system  graphics 4096000 2020-01-14 11:00 dump_RGBA8888_800x1280_1769.dat
-rw-------  1 system  graphics 4096000 2020-01-14 11:00 dump_RGBA8888_800x1280_1770.dat
-rw-------  1 system  graphics 4096000 2020-01-14 11:00 dump_RGBA8888_800x1280_1771.dat
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 其他 display HAL 代码介绍

某些平台除了基础必备的 hwc-hal 代码外，还有一些如 hdmi、cvbs 热插拔、hdmi 的 edid 解析及 cec 处理、多显策略等拓展功能代码，基于模块化的设计，这些功能的实现代码分布在其他的 SO 库中，且并非全部产品都会使用。

3.1 CEC hal

代码位于：android/hardware/aw/display/cec-hal
此部分代码用于支持 HDMI 的 CEC 功能，除非产品支持 CEC，否则该部分代码不会使用。

3.2 显示输出配置

代码位于：android/hardware/aw/display/configs
其中 dispconfigs 子目录为各配置文件的存放目录，配置文件用于对显示输出接口及分辨率、主副显映射等信息进行配置。除非产品仅仅支持 LCD 单显，否则会使用到该部分代码。

默认是通过ro.product.vendor.device属性来找配置文件，当不存在时使用 config.json 配置的文件。举例配置如下：

```
{
  "DisplayEngineVersion": 2,
  "DisplayNumber": 3, /*Display0, Display1, Display2 */

  "PrimaryDisplay": ["Display0"],
  "ExternalDisplay": ["Display1", "Display2"],

  "Display0": {
    /* Available interface type: LCD/HDMI/CVBS */
    "InterfaceType": "LCD",
    "DisplayEnginePortId": 0,
    "OverrideFramebufferSize": 1,
    "FramebufferWidth": 1280,
    "FramebufferHeight": 800,
    "DpiX": 213,
    "DpiY": 213,
    "HotplugSupported": 0,
    "AfbcSupported": 0,
    "HardwareRotateSupported": 0,
    "BandwidthLimited": 338800,
    "DefaultOutputMode": 0
  },
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
  "Display1": {
    /* Available interface type: LCD/HDMI/CVBS */
    "InterfaceType": "HDMI",
    "DisplayEnginePortId": 1,
    "OverrideFramebufferSize": 1,
    "FramebufferWidth": 1280,
    "FramebufferHeight": 720,
    "DpiX": 160,
    "DpiY": 160,
    "HotplugSupported": 0,
    "AfbcSupported": 0,
    "HardwareRotateSupported": 0,
    "BandwidthLimited": 338800,
    "DefaultOutputMode": 0
  },

  "Display2": {
    /* Available interface type: LCD/HDMI/CVBS */
    "InterfaceType": "CVBS",
    "DisplayEnginePortId": 1,
    "OverrideFramebufferSize": 1,
    "FramebufferWidth": 1280,
    "FramebufferHeight": 720,
    "DpiX": 160,
    "DpiY": 160,
    "HotplugSupported": 0,
    "AfbcSupported": 0,
    "HardwareRotateSupported": 0,
    "BandwidthLimited": 338800,
    "DefaultOutputMode": 0
  }
}
```

重点说明一些常用的可修改配置项，其他不建议修改。

- PrimaryDisplay: 只能引用后边列出的 Display0、Display1、Display2，用于配置默认安卓主显以及副显，若支持热插拔，可同时配置多个。
- ExternalDisplay: 类似于 PrimaryDisplay，用于配置按安卓副显。
- InterfaceType: 接口类型，只能为 LCD/HDMI/CVBS 之一。
- DisplayEnginePortId: 绑定到硬件 DE 的序号, 一般不修改。
- OverrideFramebufferSize: 1 表示使用 FramebufferWidth 及 FramebufferHeight 作为 UI 分辨率。
- HotplugSupported：是否支持热插拔。

3.3 显示动态配置接口实现

代码位于：android/hardware/aw/display/libdisplayconfig、 android/hardware/aw/display/interfaces
显示的一些支持动态配置的设置项在此完成支持，主要包括如亮度对比度饱和度等显示效果调整以及显示切边，HDMI、CVBS 的输出分辨率等，除非产品支持 HDMI 或 CVBS，否则不会使用到该部分代码。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.4 PQ 守护进程

代码位于：android/hardware/aw/display/pq
用于配合显示驱动与 pc 端画质调试工具，实现显示画质调整功能，一般全部产品均支持。

3.5 hdmi edid 库

代码位于：android/hardware/aw/display/libedid
主要用于实现 hdmi edid 的读取以及解析。

3.6 displayd

代码位于：android/hardware/aw/display/libdisplayd
主要用于对显示输出设备的热插拔、输出模式、多显策略等进行管理，需要设备支持 HDMI、CVBS 等热插拔输出接口，否则该部分代码不会使用到。

广州星翼电子科技有限公司zdyzlinux

4 Android 显示性能调试简介 - systrace

systrace 是 Android 平台提供的一款工具，用于记录短期内的设备活动。该工具会生成一份报告，其中汇总了 Android 内核中的数据，例如 CPU 调度程序、磁盘活动和应用线程。这份报告可帮助我们了解如何以最佳方式改善应用或游戏的性能。

systrace 工具用于显示整个设备在做些什么，不过也可用于识别应用中的卡顿。

systrace 的系统开销非常小。

4.1 使用方法

4.1.1 离线抓取

Android 系统默认包含了atrace工具，可以直接在设备端直接抓取 trace 数据，便于问题现场分析；串口终端或者 adb shell 终端直接执行下述命令，即可抓取图显相关 trace event：

```
su
setenforce 0
echo 1 > /sys/kernel/tracing/events/disp_trace/enable
atrace -b 8192 -t 10 -o /data/trace.dat gfx sched freq idle am wm view video
```

> echo 1 > /sys/kernel/tracing/events/disp_trace/enable
> AW disp2 驱动增加了部分平台独有的事件信息，主要用于性能、帧率等问题分析；上述命令的作用就是打开 disp_trace event。

上述命令生成的/data/trace.dat即为 systrace 的原始数据，可通过 Android SDK 自带的 systrace.py 转换为 HTML 报告：

```
systrace.py --from-file=trace.dat -o trace-report.html
```

4.1.2 查看支持的事件

通过命令atrace --list查看设备所支持的 trace event；显示领域较为关注的事件包括：

- gfx: 图显通路相关，显示问题排查必选项
- view：view system

- wm：window manager
- video：视频通路相关
- sched：CPU 调度相关

```
a523-pro:/ # atrace --list

      gfx - Graphics
    input - Input
     view - View System
  webview - WebView
       wm - Window Manager
       am - Activity Manager
       sm - Sync Manager
    audio - Audio
    video - Video
   camera - Camera
      hal - Hardware Modules
      res - Resource Loading
   dalvik - Dalvik VM
       rs - RenderScript
   bionic - Bionic C Library
    power - Power Management
       pm - Package Manager
       ss - System Server
 database - Database
  network - Network
      adb - ADB
 vibrator - Vibrator
     aidl - AIDL calls
    nnapi - NNAPI
      rro - Runtime Resource Overlay
      pdx - PDX services
    sched - CPU Scheduling
      irq - IRQ Events
      i2c - I2C Events
     freq - CPU Frequency
     idle - CPU Idle
     disk - Disk I/O
      mmc - eMMC commands
     sync - Synchronization
    workq - Kernel Workqueues
memreclaim - Kernel Memory Reclaim
regulators - Voltage and Current Regulators
binder_driver - Binder Kernel driver
binder_lock - Binder global lock trace
 pagecache - Page cache
    memory - Memory
   thermal - Thermal event
```

4.2 信息解读

4.2.1 CPU 运行状态

CPU 运行状态包括以下两种:

- CPU 任务信息
- CPU 频率信息

通过 CPU 运行状态信息可以大致了解进程的调度情况，能够直观地观察到由于调度引发的性能问题。

> 为观察 CPU 运行状态，atrace 命令应该带上sched以及freq两个 TAG。

图 4-1: CPU Freq Info

4.2.2 进程运行状态

> 为观察进程运行状态，atrace 命令应该带上 sched TAG。

广州星翼电子科技有限公司zdyzlinux

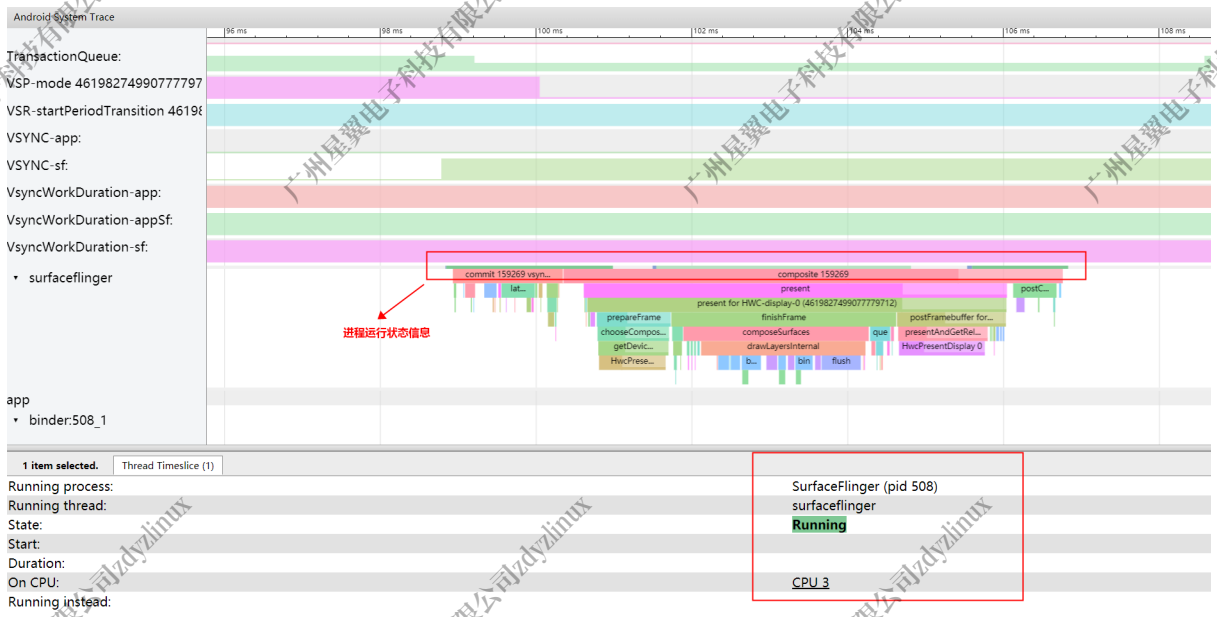

图 4-2: Process running state

进程状态是以颜色进行区分的，鼠标选中可以看到更多的细节：

颜色	进程状态	备注
绿色	正在运行	线程正在完成与某个进程相关的工作或正在响应中断
蓝色	可运行	线程可以运行但目前未被调度执行
白色	休眠	线程没有可执行的任务，可能是因为线程在遇到斥锁定时被阻止
橙色	不可中断的休眠	线程在遇到 I/O 操作时被阻止或正在等待磁盘操作完成
紫色	可中断的休眠	线程在遇到另一项内核操作（通常是内存管理）时被阻止

4.2.3 SurfaceFlinger

针对图显类问题，主要解读 SurfaceFlinger 进程下的相关信息：

1. VSYNC 信号是否符合预期

- 周期是否与接口输出帧率匹配
- 是否与硬件 VSYNC 对齐

2. SurfaceFlinger 合成送帧的均匀度和周期
3. GPU/display fence 状态
4. 图层 Queued-Buffer 数量
5. 调用栈里的各函数耗时
6. GPU completion

4.2.3.1 VSYNC

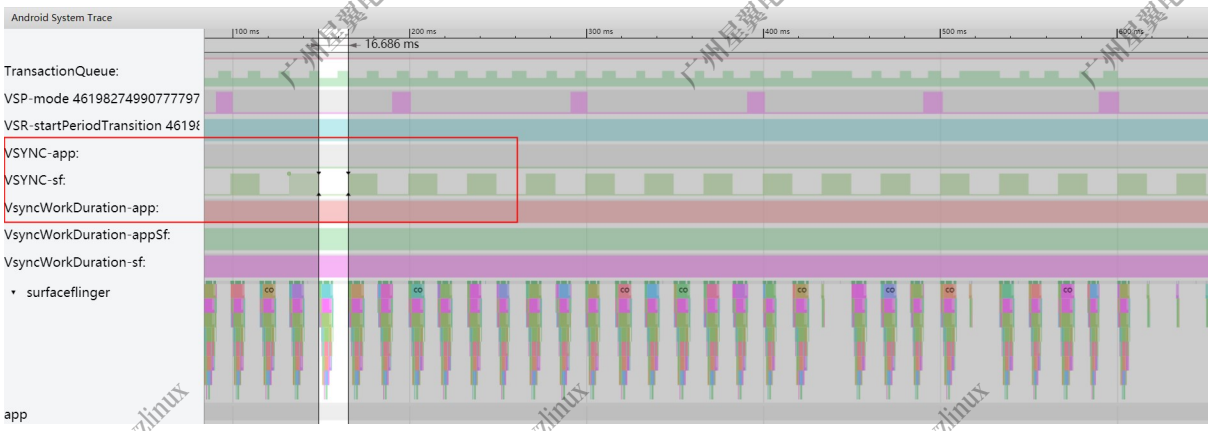

图 4-3: SurfaceFlinger VSYNC

VSYNC 周期应该与输出接口的刷新周期相符，通常为 16.67ms（60Hz 刷新率）。

VSYNC 并不是时刻存在的；当 SurfaceFlinger 不需要合成的时候（APP 没有更新 Buffer 给 SurfaceFlinger），会停止 VSYNC-SF，这是正常现象；

因此，VSYNC-SF 是按需出现的！当存在 VSYCN-SF，但是 SurfaceFlinger 没有进行合成送帧操作，通常意味着有问题！

如下图红色框框部分，VSYNC-SF 触发主线程工作，但是 isFencePending() 返回真进而跳帧：

> 这通常意味着显示通路上某个阶段耗时异常或者 disp2 驱动发生了跳帧。

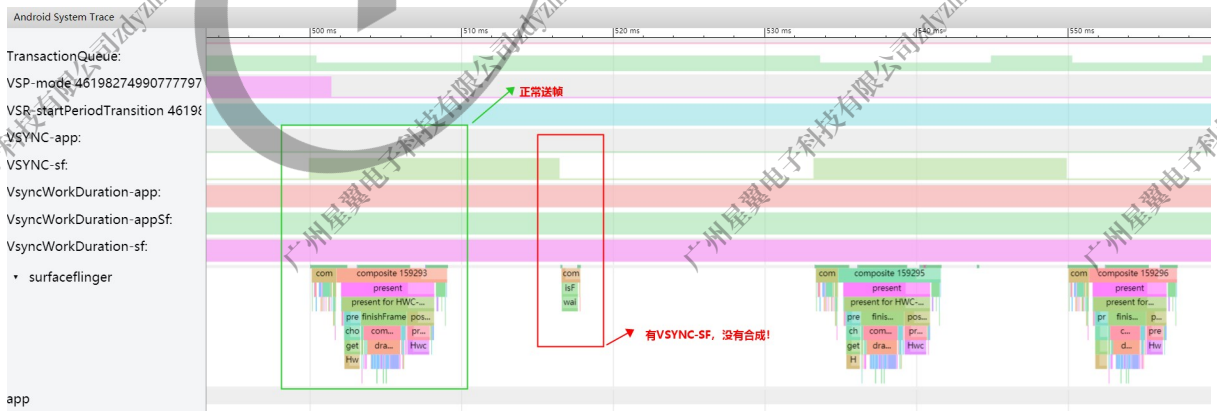

图 4-4: Fence not signal

4.2.3.2 fence state

主要关注 Fence 的 unsignal 状态是否持续时间过长；对于 SurfaceFlinger 进程，我们主要关注 GPU fence，它表征着 GPU 何时完成渲染。

在 systrace 上，GPU fence 呈现为：GPU completion。

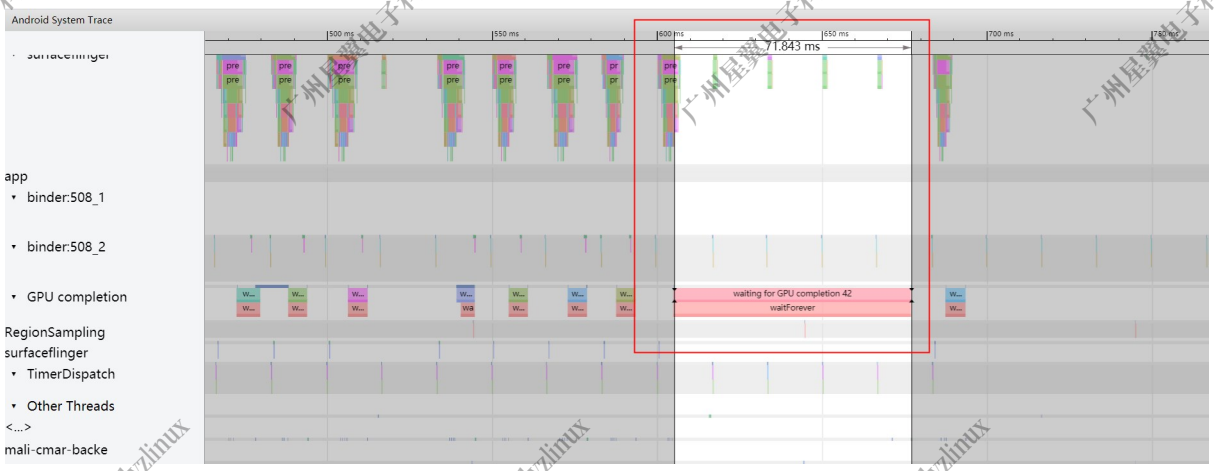

图 4-5: GPU completion

如上图的异常情况，GPU completion 42耗时 71+ms。

4.2.3.3 Queued-Buffer 数量

Queued-Buffer 数量表征着图层是否存在待合成的 Buffer；如果 Queued-Buffer 数量为 0，意味着 SurfaceFlinger 在等待应用绘制完成。

像视频播放卡顿类型问题，首先需要确认对应的 SurfaceView 是否存在 Queued-Buffer。

4.2.4 composer

composer 进程也是图显领用重点关注的，systrace 提供了以下信息：

- Queued-Frame 数量
- acquire fence 耗时
- display release fence 耗时
- G2D 旋转耗时
- commit 耗时

版权所有 © 珠海全志科技股份有限公司。保留一切权利

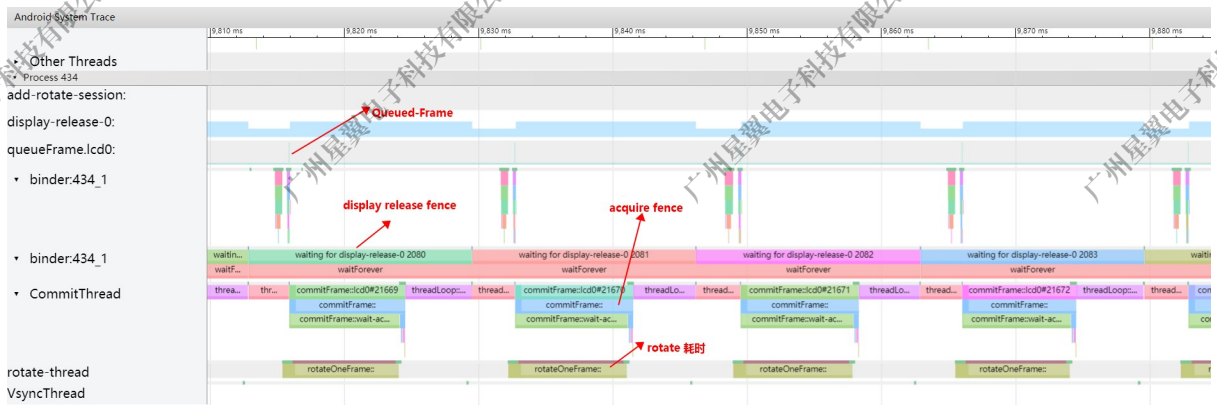

图 4-6: systrace composer info

4.2.5 disp_trace

AW 在 disp2 驱动增加了部分 trace 信息，用于观察：

- vsync-irq：vsync 硬件中断
- frame-skip：跳帧标志

可以方便定位由于 vsync-irq 调度延迟导致的丢帧问题。

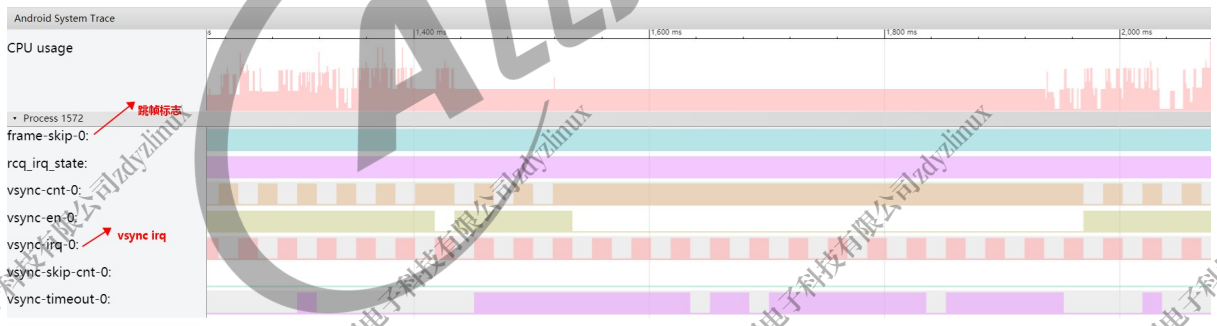

图 4-7: disp2 trace

著作权声明

版权所有 ©2023 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。